SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 3 – DEBUGGING & OPTIMISATION TASK

Course Code:	CSA0618	**Course Title:**	Design and Analysis of Algorithms
CO Assessed:	CO2 – Manipulation (BL1, BL2, BL3)	**Assessment:**	Assessment Tool 3 – Debugging & Optimisation Task
Weightage:	10% of CIA	**Total Marks:**	100
CO2 Statement:	*Apply Brute Force technique to solve sorting, searching, Closest-Pair and Convex-Hull problems (BL1, BL2, BL3)*		
Student Name:	Dhanush Kumar R	**Reg. No.:**	192411215
Section / Batch:	BE CSE	**Date:**	24.07.2026

Instructions to Students

- Answer the following question. Total Marks: 100.
- Carefully analyze the given program/code segment before identifying errors.
- Clearly identify logical, syntactical, and performance-related issues in the code.
- Apply systematic debugging techniques to correct the errors effectively.
- Optimize the given solution by improving efficiency, reducing unnecessary computations, or enhancing time complexity wherever possible.
- Provide proper justification for all corrections and optimization decisions.
- Write clean, structured, and well-documented code with appropriate comments.
- Maintain clarity, logical reasoning, and technical accuracy throughout the solution.

Question Type	Focus Area	Questions
Debugging & Optimisation Task	Identify errors, debug programs, and optimize algorithm efficiency using appropriate techniques	Q1

SECTION A – DEBUGGING & OPTIMISATION TASK

Q1. Debugging Selection Sort Code

```
def selection_sort(arr):
    n = len(arr)
    for i in range(n):
        min_idx = i
        for j in range(i+1, n):
            if arr[j] > arr[min_idx]:   # ERROR
                min_idx = j
        arr[i], arr[min_idx] = arr[min_idx], arr[i]
    return arr
```

Corrected Code

```
def selection_sort(arr):
    n = len(arr)

    for i in range(n - 1):
        min_idx = i

        # Find smallest element
        for j in range(i + 1, n):
            if arr[j] < arr[min_idx]:
                min_idx = j

        # Swap only if needed
        if min_idx != i:
            arr[i], arr[min_idx] = arr[min_idx], arr[i]

    return arr

# Example
arr = [64, 25, 12, 22, 11]
print(selection_sort(arr))
```

OUTPUT
[11, 12, 22, 25, 64]

Q2. Debugging Linear Search Code

```
def linear_search(arr, key):
    for i in range(len(arr)):
        if arr[i] == key:
            return -1
    return i
```

CORRECTED CODE

```
def linear_search(arr, key):

    for i in range(len(arr)):
        if arr[i] == key:
            return i

    return -1


arr = [10,20,30,40,50]
```

```
print(linear_search(arr,30))
print(linear_search(arr,100))
```

OUTPUT
2
-1

Q3. Debugging Closest Pair Code

```
import math

def closest_pair(points):
    min_dist = 0

    for i in range(len(points)):
        for j in range(i+1,len(points)):
            dist = math.sqrt((points[i][0]-points[j][0])**2 +
                        (points[i][1]-points[j][1])**2)

            if dist > min_dist:
                min_dist = dist

    return min_dist
```

CORRECTED CODE

```
import math

def closest_pair(points):

    min_dist = float('inf')

    for i in range(len(points)):
        for j in range(i+1, len(points)):

            dist = math.sqrt(
                (points[i][0]-points[j][0])**2 +
                (points[i][1]-points[j][1])**2
            )

            if dist < min_dist:
                min_dist = dist

    return min_dist
```

```
points = [(1,2),(4,6),(5,5),(10,12)]

print(closest_pair(points))
```

Q4. Debugging Convex Hull Logic

```
def orientation(p, q, r):
    return (q[1]-p[1])*(r[0]-q[0]) - (q[0]-p[0])*(r[1]-q[1])
```

CORRECTED CODE

```
def orientation(p, q, r):

    val = ((q[0]-p[0])*(r[1]-q[1]) -
           (q[1]-p[1])*(r[0]-q[0]))

    if val == 0:
        return 0

    elif val > 0:
        return 1      # Clockwise

    else:
        return 2      # Counter-clockwise
```

Q5. Debugging Binary Search Code

```
def binary_search(arr, key):

    low, high = 0, len(arr)-1

    while low < high:

        mid = (low+high)//2

        if arr[mid]==key:
            return mid

        elif arr[mid] < key:
            high = mid-1

        else:
            low = mid+1
```

return -1

CORRECTED CODE

```
def binary_search(arr, key):

    low = 0
    high = len(arr) - 1

    while low <= high:

        mid = low + (high - low) // 2

        if arr[mid] == key:
            return mid

        elif arr[mid] < key:
            low = mid + 1

        else:
            high = mid - 1

    return -1


arr = [2,4,6,8,10,12,14]

print(binary_search(arr,10))
print(binary_search(arr,7))
```

OUTPUT

```
4
-1
```

Code of Conduct

I Dhanush Kumar/192411215 *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the Student: Dhanush Kumar R

RUBRICS FOR CALCULATION

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Identification	20%	Accurately identifies all errors and root causes with clear understanding	Identifies most errors with minor gaps	Identifies some errors but misses key issues	Unable to identify errors correctly
Debugging	25%	Applies systematic debugging techniques effectively to resolve issues	Uses appropriate debugging methods with minor errors	Limited debugging approach; requires guidance	Unable to debug or resolve issues
Optimization	20%	Optimizes code by significantly improving time complexity using efficient algorithms	Improves time complexity with minor gaps in efficiency	Limited improvement in time complexity	No improvement; inefficient approach retained
Analysis	20%	Demonstrates strong logical reasoning and clear justification of solutions	Shows reasonable analysis with some justification	Limited analysis; weak reasoning	No analytical reasoning demonstrated
Code Quality	15%	Produces clean, well-structured code with proper comments and documentation	Code is mostly clear with minor documentation gaps	Code lacks structure and proper documentation	Poorly written code with no documentation

Marks Evaluation:

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Identification	20%				
Debugging	25%				
Optimization	20%				
Analysis	20%				
Code Quality	15%				
Total Marks (100):					

Faculty In-charge	Course Coordinator	Head of Department
Name:	Name:	Name:
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____